



Produced by
Psion Teklogix Ltd

Comms User Manual

Version 1.00
July 2001
English

© Copyright Psion Teklogix Ltd, 2000

Reproduction in whole or in part, including utilisation in machines capable of reproduction or retrieval, without the express written permission of the copyright holders is prohibited. Reverse engineering is also prohibited.

The information in this document is subject to change without notice.

Some names referred to are registered trademarks.

© Copyright Symbian Ltd. 1997-1999

Part of the IMAP4 software addition is © Copyright ANT Ltd. 1998. All rights reserved.

Incorporates MPPC® compression from Hi/fn™.

Stac ®, LZS ®, ©1996, Stac, Inc., ©1994-1996 Microsoft Corporation. Includes one or more U.S. Patents: No. 4701745, 5016009, 5126739, 5146221, and 5414425. Other patents pending.

Incorporates LZS® compression from Hi/fn™.

Hi/fn ®, LZS ®, ©1988-98, Hi/fn. Includes one or more U.S. Patents: No. 4701745, 5016009, 5126739, 5146221, and 5414425. Other patents pending.

Contents

Introduction	1
About Comms	1
Starting Comms	1
The Terminal emulation screen	1
Setting up communications with remote machines	2
Comms port	2
Baud rate	2
Data bits, stop bits and parity	2
Handshaking	2
The type of Terminal emulation	3
Special codes	3
Connecting to another machine	3
Direct connection to another machine	3
Setting up HyperTerminal for connection to a netpad	4
After the connection has been established	4
Connecting via a modem link	4
Connecting via infrared	5
Stopping any type of connection	5
Sending and receiving files	5
Sending a file	5
Sending more than one file	6
Receiving a file	6
Receiving ASCII information	6
Scripts	7
The example script	7
Checking scripts	7
Running scripts	8
An introduction to the scripting language	8
How script commands are used	8
Names	9
Using names in a script	10
Storing name information using the QUERY command	10
Reserved names	10
Password protecting names	11
Variables	11

Script commands	11
Summary of the script commands	11
Glossary of script commands	13
How glossary entries are arranged	13
Syntax abbreviations	13
Glossary	14
Index	28

Introduction

This User Guide provides information about the netpad's Comms program. Comms provides terminal emulation and file transfer to other computers. You can use it to:

- Connect the netpad directly to another computer.
- Connect the netpad via a modem to access electronic mail systems and bulletin boards.



You can transfer files between the netpad and a PC more easily using PsiWin 2.x and the docking cable. This also allows file conversion between PC and netpad formats. Contact your Psion distributor.

About Comms

Comms is the netpad's communications and terminal emulation program. Use it to connect to remote computers, control modems and to use your netpad as a remote terminal when connected to a host computer.

The program has two views:

- **The Terminal emulation screen**, where you connect to and communicate with remote machines. This is the screen you see when you first start Comms.
- **The Script editor**, where you create scripts to automate parts of communications, for example logging on to a remote host.

This User Guide provides an introduction to the Terminal emulation screen and its features, and then describes how to create your own scripts. A glossary of all the commands in the Script language is included.

Starting Comms

To start Comms:

1. Make sure that the Remote link on the netpad is switched **off**. To do this, select **Link to Desktop** on the **Tools** menu in the System screen and make sure that Off is displayed in the Link line.
2. Tap the System command icon, then tap the Comms program icon.
When Comms starts, you will see the Terminal emulation screen, and an **Offline** information message will appear.

Comms communicates with other machines through either the serial or infrared port. The message in the status bar shows 'Online' or 'Offline' depending on whether a port is active or not.

- If no port is currently active, check that the Remote link is off and that the **Port active** command on the **Transfer** menu is ticked.
- To exit Comms, use the **Close** command on the **File** menu.
- To hide the status bar, remove the tick from **Show status bar** on the **View** menu.

The Terminal emulation screen

The Terminal emulation screen is where you:

- Set up the netpad for communication with a remote machine.
- Start and stop connections to remote machines.
- Type characters that are sent via the communications port to the remote machine or modem.
- View characters that you receive from the remote machine.
The characters you type on the netpad, and those you receive from remote machines, are displayed in the Terminal screen.
- Send and receive files.



Displaying the characters you type on the netpad is called “local echo”. To switch the local echo off, select [Translate codes](#) from the [Tools](#) menu, and remove the tick from the Local echo line.

Setting up communications with remote machines

Before starting a connection, make sure that the communication settings on the netpad match those of the device you are connecting to. Use [Communication settings](#) on the [Tools](#) menu, or the [Set up](#) button on the Toolbar, to adjust the following communications parameters:

- The comms port used.
- The baud rate (the speed of data transfer between the two machines).
- The number of data and stop bits.
- Parity.
- The “Handshaking” options.

Comms port

Set the Use comms port line to one of the following:

- **Cable:** to connect using the serial port at the back of the machine, e.g. if you are using the Docking cable with an adaptor to connect to a modem.
- **Infrared:** to start an infrared communications link.

Baud rate

The netpad can transfer data at speeds of up to 115200 bits per second. Select an appropriate speed in the Baud rate line of the Communication settings dialog. Make sure that the speed you have set on the netpad matches the speed setting on the remote machine, and does not exceed the maximum speed of the modem you are using.



Modern modems can sense and adjust to the speed that you are transmitting at. With older modems, you may need to specify the exact speed of the connection.

Data bits, stop bits and parity

The data bits, stop bits and parity settings determine how each character to be transmitted is converted into bits.

- **Data bits:** Set the number of data bits to the same as the setting on the remote machine.
- **Stop bits:** Set the number of stop bits to be the same as the setting on the remote machine. Usually 1 stop bit is sufficient, but with low Baud rates (300 or less), or when communicating with some mechanical devices, you may need to set 2 stop bits. Having 2 stop bits set instead of 1 never causes an error (though the transfer rate is slightly slower). However, setting 1 stop bit when you should have 2 will cause errors.
- **Parity:** Choose from none, even and odd parity. Make sure the setting is the same as the one on the remote machine.

Handshaking

Use the tick boxes on the Handshaking page of the Communication settings dialog to select which of the following handshaking protocols are used when communicating:

- XON/XOFF
- RTS/CTS
- DSR/DTR
- DCD

Usually, when communicating via a modem, XON/XOFF, DSR/DTR and RTS/CTS handshaking are set, although this may vary according to the setup on the remote computer and your modem manufacturer's guidelines.

- Handshaking is sometimes termed 'flow control'.
- For Hardware flow control, tick the RTS/CTS box.

The type of Terminal emulation

Comms can emulate a teletype terminal or a VT100 terminal.

Select **Terminal emulation** on the **Tools** menu.

In the Terminal to emulate line, choose TTY for a teletype, or VT100.

- You can adjust the width and height of the screen by setting the number of characters for each in the Terminal emulation dialog. If you set a greater width or height than can be displayed at the current zoom level, use the scroll bars to view other areas of the screen.
- To hide the scroll bars, remove the tick from **Show scroll bars** on the **View** menu.
- Use **Clear screen** on the **Edit** menu to remove all the characters from the display.

Special codes

Different computers may require different codes to be sent when the **Enter** key is pressed. Comms is initially set to add a line feed character to both incoming and outgoing carriage returns.

To change the special codes:

1. Select **Translate codes** on the **Tools** menu,
2. Use the Add LF to incoming CR and Add LF to outgoing CR lines.

Connecting to another machine

There are three ways you can connect your netpad to a remote machine using Comms:

- Direct connection to another machine.
- Connection to another machine via a modem link.
- Connection via infrared.

Direct connection to another machine

To connect the netpad directly to, for example, a PC:

1. Connect the cable to the serial port of the netpad, and to a COM port on the PC. You may already have this set up if you use PsiWin 2. Make a note of the COM port on the PC that the cable is connected to.
2. Make sure that neither machine is running other software that uses the communications ports.
 - In the case of the PC, make sure that you are not currently running PsiWin 2.
 - Make sure the netpad has the Remote link switched off, and is not trying to carry out an infrared transfer.
3. Start Comms on the netpad, and run a suitable Terminal application on the other machine. If you use Windows 95, this may be HyperTerminal.
4. Set both pieces of software to use the appropriate ports, and to use identical settings for Baud rate, Data bits, Stop bits, Parity and Handshaking. The two machines are now ready to exchange characters and files.

Setting up HyperTerminal for connection to a netpad



The first time you use HyperTerminal to connect to the netpad, you will need to create a "new connection".

To set up HyperTerminal:

1. Select **HyperTerminal** from the **Programs|Accessories** branch of the **Start** menu, and double-click on the **Hypertrm** icon. Select an icon and give a name to the connection, e.g. My netpad.
2. In the **Phone number** dialog, make sure the **Connect using** line is set to the COM port to which you have connected the cable.
3. Use the **Properties** dialog to specify identical communication settings to those on the netpad. In the Flow control line, Hardware is equivalent to having the RTS/CTS box ticked on the handshaking page of the Communication settings dialog in Comms.

After the connection has been established

After you have established a connection, the characters you type on the netpad appear in the PC's display, and characters typed on the PC appear in the Comms Terminal emulation screen. This connection allows you to send files between the machines: see the *Sending and receiving files* section later in this manual for details.



You can use the **Copy** and **Paste** commands on the **Edit** menu to copy and paste text to and from other netpad programs. When you paste text in the Terminal emulation screen, it is sent to the remote machine: this is often called "pasting to a host".

When you've finished the communications session, you may want to:

- Save the settings on the other machine. In HyperTerminal this is done using **Save** on the **File** menu.
- Save the communications settings on the netpad using **Save settings as** on the **File** menu, giving the file an appropriate name, e.g. netpad to PC.

You will then be able to start communications between the machines again quickly, using **Load settings** on the **File** menu in Comms, and by clicking on the appropriate icon in the HyperTerminal folder.

Connecting via a modem link

Connecting the netpad to a remote machine via a modem link is very similar to connecting directly with a PC, except that the signals between the two machines travel via a telephone line with a modem at each end. You must make sure:

- The Baud rate you use does not exceed the maximum speed that the two modems are capable of handling.
- You correctly initialise the modem that the netpad is connected to.

To start a connection:

1. Connect the netpad to the modem using a serial cable and a modem adaptor lead - contact your Psion distributor for details. Connect the modem to a telephone socket.
2. Set the communication settings in Comms that are appropriate to the modem's maximum speed, and identical to the remote machine's settings.
3. Start communications with the modem by typing

AT

The modem should return OK, which appears in the Terminal emulation screen. If it doesn't, make sure all cables are connected correctly and the communication settings are appropriate to the modem according to the manufacturer's instructions.

4. Type the modem initialisation string appropriate to the modem. This may be ATZ or AT&F, but you should check the modem manual for further details.
5. Type ATDT followed by the telephone number of the remote modem you are connecting to (including any dial-out prefix you need to add).

When the other modem answers, you normally see a **CONNECT** message in the Terminal window, and can begin to log in according to the remote machine's requirements. This may require pressing the **Enter** key a few times to get a prompt (a line of text sent by the remote machine, for example `user name :` that waits for a command or a response).

When communicating with a remote host, you may find you receive characters too quickly to read. Use **Pause** on the **Transfer** menu to temporarily stop incoming information. Selecting **Pause** again resumes the transfer.



If you cannot use any kind of handshaking, you may begin to lose data if the netpad display is paused for some time while the other machine is sending information. To avoid this, try setting a lower Baud rate that allows you to read the incoming information more easily.

Connecting via infrared

You can communicate with another netpad, an infrared-enabled PC or an infrared-enabled modem using Comms and the netpad's infrared port.

To communicate using infrared:

- Select **Communication settings** on the **Tools** menu and set the comms port to Infrared.
- Make sure the other machine is ready to communicate with the netpad via infrared, and then continue setting up the connection as you would if you were using a cable (see *Direct connection to another machine*).



Because normal infrared file transfer between two netpad machines does not allow you to send and receive multiple files, you may want to connect directly to another netpad using a cable. You can then use the YModem (batch) protocol in Comms to send and receive more than one file at a time. See the *Sending and Receiving files* section.

Stopping any type of connection

- Use **Hangup** on the **Transfer** menu to close the current connection. If you are connecting via a modem link, the modem will drop the telephone line.

Sending and receiving files

After you have established communications with another machine, you can use Comms to send and receive files. The other machine must be able to use at least one of the following protocols:

- ASCII
- XModem
- YModem (batch)

The ASCII protocol is adequate for plain text transfers. If you can, use XModem or YModem (batch) protocols to transfer files, because they include error checking. The YModem (batch) protocol also allows you to send or receive more than one file at a time.



When transferring files, it is important to use the same protocol as the remote machine, otherwise information won't be interpreted correctly. You set the protocol using the Protocol lines of the Send file and Receive file dialogs.

Sending a file

To send a file to a remote machine after a connection has been established:

1. Select **Send file** on the **Transfer** menu, or tap the **Send file** button on the Toolbar.
2. Select the transfer protocol in the Send file dialog. Make sure it is one that the remote machine supports and is prepared to receive.
3. Use the Name, Folder and Disk lines to locate the file you want to send.

4. Press **OK** on the netpad and prepare the remote machine to receive a file. If possible, perform these actions at the same time.

Sending more than one file

To send more than one file:

1. Create a folder specifically for the files you want to send, place all the relevant files in it.
2. Select **Send file** on the **Transfer** menu.
3. Select the YModem (batch) protocol, then select the folder containing the files you want to send in the Folder line.
4. Press the Tab key in the Name line, then drag the pen to select all the files in the folder. Press **Enter**.
5. Press **OK** and prepare the remote machine to receive a file.

Receiving a file

To receive a file from a remote machine after a connection has been established:

1. Select **Receive file** on the **Transfer** menu, or tap the **Receive file** Toolbar button.
2. Select the transfer protocol in the Receive file dialog. Make sure that it is the one the remote machine will use to send the file.
3. Choose a folder for the file to be saved in. If you are receiving a file using either the ASCII or XModem protocol, specify a filename. (YModem transfers take the filename from the sending machine.)
4. Press **OK** on the netpad and instruct the remote machine to send the file. If possible, perform these actions simultaneously.

To receive more than one file:

Follow the steps above, but the YModem (batch) protocol in step 2. Tell the other machine to send a number of files using the YModem protocol. At the end of the file transfer you will have a number of files in the target folder.



When receiving .OPO program files and other files that normally have extensions in their filenames, include these extensions in the names you give each of the received files, otherwise the netpad may not be able to run them correctly.

While sending or receiving a file, a dialog shows the progress of the transfer. When the transfer has finished, Comms displays a **File transfer succeeded** message and returns to the Terminal emulation screen.



To cancel a transfer at any time, tap the **Cancel** button in the file transfer dialog.

If you set the netpad to receive a file, it “times out” (terminates the connection when nothing has happened for a given period) if a connection is not made within a certain time. However, when transmitting, it will not time out, and will continue waiting for a connection.

Receiving ASCII information

If you are receiving information from a remote machine as plain ASCII characters, e.g. when reading e-mail on a server, you can save this information to a file in order to have a record.

To save ASCII information:

1. Select **Capture to file** on the **Transfer** menu and enter the file a name.
 - If you want the new information to be added to the end of an existing file, **enter** the name of the existing file and tick the Append box.
 - If you want incoming control characters to be recorded in angled brackets, e.g. ‘<13>’ for a carriage return, tick the Debug box.
2. To end the capture, deselect the **Capture to file** menu option.

Captures are stored as plain text files. You can view a plain text file in Word, by using the **Import text file** command on the **More** cascade from the **File** menu.

If you have problems transferring information with no protocol, try setting both computers to use a lower Baud rate.

Scripts

The Comms program contains a Script editor that allows you to create lists of instructions to automate procedures such as logging on to a remote computer. Scripts allow you to control all the aspects of a connection that you would normally handle in the Terminal emulation screen, such as communication settings and file transfers.

This section provides an introduction to creating and working with scripts, and is followed by a full glossary of the scripting language commands.

To create a new script:

1. Select [Create new script](#) from the [Scripts](#) menu. Enter a name and location for the script and press **OK**.



Script names must have a '.scr' extension. When you save your script, Comms will add '.scr' to the end of its name.

2. Comms opens the script editor.
3. Enter your script.
4. Tap [Save](#) to return to the main Comms view.

When you are creating a new script, e.g., a connection to a BBS system, it can be useful to first record the logging in process using [Capture to file](#) on the [Transfer](#) menu. This records the prompts sent to you by the remote machine, and can then be used as a starting point for creating the script.

To edit an existing script:

1. Select [Open](#) on the [Scripts](#) menu, and choose the script you want to work on. Press **OK**.
2. When you have finished editing the script, tap [Save](#) or use [Save as](#) to give the new version a different name.

Comms returns to the Terminal emulation screen.

The example script

The netpad comes with an example script using a wide range of the features available in the scripting language.

- **To access the script:** Select [Create standard files](#) on the [Tools](#) menu. Select a folder for Comms to place the example script in, and press **OK**.

The script is called 'Cix.scr', and is designed for connection to the CIX (Commercial Internet Exchange) service, although it can easily be adapted for other dial-up services.

- **If you have a CIX account:** When you first run the script, you are prompted for the telephone number you dial to, as well as your user name and password details. This information is stored in the Cix.nam names file. A menu is displayed with a number of options, e.g. 'Download mail and log off' and 'Change logon details'.
- **If you do not have a CIX account:** You can still edit and adapt the supplied script to suit the dial-up service you use, using the [Open script](#) command on the [Scripts](#) menu.

Checking scripts

Comms has a built-in syntax checker that searches for bugs in your scripts. It is a good idea to use this before trying to connect, as spotting any bugs in advance can save a lot of time.

- **To check your script:** select [Syntax check](#) on the [Scripts](#) menu and specify the script you want to check.

If there are any errors, the Script editor positions the cursor at the appropriate point in the script and informs you of a problem.

Running scripts

After you have created a script, run it by selecting **Start Script** from the **Scripts** menu, and choosing the appropriate script in the Start script dialog.

While a script is running, its name is displayed in the status bar at the bottom of the Terminal emulation screen.

- **To stop a script**, select **Stop script** from the **Scripts** menu.

When a script finishes running, the Terminal emulation screen is displayed. You can run another script simply by selecting **Start Script** again. You can also instruct one script to start another when it finishes. For information on how to do this, see the glossary entry on the **CALL** command later on.



You cannot start a script running while the Comms port is inactive. Make sure that **Port active** on the **Transfer** menu is ticked in the Terminal emulation screen.

An introduction to the scripting language

The language used in scripts is made up of commands that are similar to ordinary English words. This section provides an introduction to some important aspects of the script language. The following section guides you through all the script commands.

How script commands are used

A short script might look like this:

```
STATUS "Uploading test file"
UPLOAD "C:\Documents\anyfile", "Xmodem"
EXIT
```

This script uses three commands:

- **STATUS**, which displays an information message in the bar at the bottom of the screen;
- **UPLOAD**, which sends a file to the remote machine,
- **EXIT**, which stops the script running, returning control to the Terminal emulation screen.

Commands can be entered in upper or lower case, or a mixture of both. To combine more than one command on a line, separate them with a colon, e.g.

```
INFO "Receiving test transmission" : DOWNLOAD "C:\Templtrashfile" , "XModem" : EXIT
```

The total length of a line must not exceed 255 characters.



In general, finish a script with a carriage return (an extra blank line at the end of the script).

Labels

Scripts are split up into sections marked by "labels". A label, e.g. 'mylabel:' denotes the start of a section of script. Use labels to split up scripts into manageable sections, and to enable the script to jump directly to a given area by using the **GOTO** command. For example, in the following script, the first status message is never displayed, because the **GOTO** command jumps straight to 'mylabel:'

```
GOTO mylabel:
STATUS "Didn't jump to mylabel"
mylabel:
STATUS "Jumped to mylabel"
```



To learn how to repeat the instructions given in a label a specific number of times, see the **SET** and **REPEAT...UNTIL** commands in the glossary of commands later in this manual.

End-commands

Some commands, e.g. **SETUP**, **IF** and **QUERY**, require an “end-command” to tell the machine that the instructions they refer to are finished. For example,

```
SETUP
  Baud=57600
  parity=Even
  handshake=XONXOFF
ENDSETUP
```

Commands that require ending are indicated in the glossary of script commands later in this document.

Text strings

Strings are sequences of characters between quotes. The **SEND** command sends a string through the serial port. For example,

```
SEND "ATZ"
```

This sends the characters **A**, **T** and **Z**, a modem initialisation string. You can insert control characters in a string by putting them in angled brackets, for example,

```
SEND "guest<013>"
```

sends a carriage return character after the text “**guest**”. Control character codes can also be entered in hexadecimal form, prefixed with a \$ sign. For example,

```
SEND "bye<$1B>"
```

Puts the control character code 27 (the ‘Esc’ control) at the end of the **SEND** string.

Filename and folders

When you are receiving a file using the XModem protocol or ASCII, you must specify a name for the file to have on the netpad. To do this, either:

- Simply enter the filename, for example,

```
DOWNLOAD "LetterIV", "XModem"
```

This places the downloaded file in the Documents folder.

- Specify the full “path name”, including the folder and disk, e.g.

```
DOWNLOAD "C:\Documents\Letters\LetterIV", "XModem"
```

Use this method to download files directly to a memory disk, e.g.

```
DOWNLOAD "D:\Publications\Frontcover", "XModem"
```



Make sure the name you give to the file you are downloading is not the same as that of another file within the same directory, otherwise you may overwrite the existing file.

When you are receiving files using the YModem (batch) protocol, the sending machine transmits the filename — there is no need to specify one. Simply use a “blank text” (no characters) string in the command, e.g.

```
DOWNLOAD "", "YModem (batch)"
```

The file will be saved in the Documents folder.

Names

Comms can store information you use frequently, such as passwords or telephone numbers, as “names” in a file. You can then insert a name in a **SEND** or **SENDWAIT** commands which Comms will substitute with the stored information.

To store information as a name:

Select **Set up names** on the **Names** menu in the Terminal emulation screen, and tap **Add**.

Enter the name, e.g. “pass”, and then enter the value, e.g. a password.

- The text represented by the name can be up to 64 characters long.
- Names can be up to sixteen characters long, and must start with a letter.
- Names can only be used with the **SEND** and **SENDWAIT** commands.

After you have created a set of names, save them using the [Save names as](#) command on the [Names](#) menu. You can then use [Load names](#) to retrieve the specific set of names that you need at any one time. In general, you should only need to use one names file, even if it contains more names than you will ever use at one time.

- To change the information for a name, use the [Set up names](#) command, and edit the Value line for the relevant name.

Using names in a script

To use named information in a script, you need to instruct the machine to load the correct names file using the `loadnames` command, e.g.

```
LOADNAMES "D:\Comms\Comms.nam"
```

You can then use the information in this file in either a `SEND` or `SENDWAIT` command. To do this, add a \$ sign to the name. For example, to send the information named "pass1", use the line:

```
SEND pass1$
```

You can send more than one name at once - or mix different kinds of information - by using the `&` operator. For example,

```
SEND username$ & pass$ & "mail"
```

- If you alter named information in the course of a connection, or create new named information using, say, the [QUERY](#) command, you can then use the [SAVENAMES](#) command to preserve the changes, e.g.

```
SAVENAMES "C:\Comms\Names\BBSnames.nam"
```

Storing name information using the QUERY command

You can create names while the script is running, using the [QUERY](#) command. The following example prompts the user with a dialog asking for information which is then stored under the names "no", "init", "name" and "pass".

```
QUERY "Dial-up details"  
  "BBS telephone number" , no$  
  "Modem initialisation string" , init$  
  "Username" , name$  
  "Password" , pass$  
ENDQUERY
```

Reserved names

There are three "reserved" names (i.e. not free to use for any information), which can only be used for special kinds of information. They are:

```
sys_echo$  
sys_incoming_crlf$  
sys_outgoing_crlf$
```

These are the scripting language equivalents of the [Translate codes](#) options on the [Tools](#) menu in the Terminal emulation screen. They toggle the "local echo" feature, and control whether or not a line feed instruction is added to each incoming and outgoing carriage return.

The following line switches the local echo function on, which allows you to see the characters you are sending on the home screen. Setting a value of 0 switches it off.

```
sys_echo$=1
```

The following line tells the netpad to start a new line each time the remote machine sends a carriage return. Setting a value of 0 switches it off.

```
sys_incoming_crlf$=1
```

The following line tells the remote machine to start a new line each time you send a carriage return from the netpad. Setting a value of 0 switches it off.

```
sys_outgoing_crlf$=1
```

Password protecting names

You may want to protect confidential information stored as a name by giving the names file a password.

To do this:

1. Select **Password** on the **Names** menu, then enter and confirm your password.
2. Select **Save names** on the **Names** menu. You will now have to enter the password to access the Names dialog.
- **To clear a password:** select **Password**, enter the current password, and then enter a zero-length password in the Set password dialog.



You can still run scripts that use names without entering the password. If you want to be prompted every time a script that uses password protected names runs, use the **LOADNAMES** command. The following example prompts for a password if the file being loaded is password-protected.

```
LOADNAMES "C:\BBSstuff\Comms.nam"  
SEND init$ & phone$
```

Variables

A variable is a name that can be used to store and represent different information. For example, **username\$** could be used to represent the information "Anton", "Sam" or "5" etc, depending on what has been stored under that name. There are two types of variables in the scripting language:

- **Non-persistent variables:** have no \$ after them, e.g. "**phoneno**". Information stored in this kind of variable is lost when the script stops running.
- **Persistent variables:** are followed by a \$. Information stored in this kind of variable can still be edited and saved after the script has stopped running, using the **Set up names** and **Save names as** commands on the **Names** menu.

Script commands

This section provides a summary of the script commands according to type, followed by a detailed glossary of commands and their usage.

Summary of the script commands

Port and handshaking control

```
CONNECT  
HANGUP  
RESET  
SETUP  
LINE
```

Sending and testing for character strings

```
ON REPLY  
SEND  
SENDWAIT  
SENDBREAK  
WAIT
```

string operations

```
ASC  
LEFT  
MID
```

RIGHT
&

File handling

APPEND
CAPTURE
DOWNLOAD
EXISTS
UPLOAD
LOADNAMES
SAVENAMES
OPEN
AS
CLOSE
READ
WRITE
EOF

User information & interaction

ALERT
BEEP
INFO
MENU
QUERY
STATUS

Program control

CALL
EXIT
FORGET
GOTO
ON
SET
IF
OR
AND
ELSE
ELSEIF
THEN
WHILE
DO
REPEAT
UNTIL

Other

//
DRAIN
VERSION

Glossary of script commands

This section contains an alphabetical list of all the commands available when creating scripts. Examples are given for each command. You may also find it useful to look at the example script given at the end of the list.



Some commands, such as **SETUP** and **MENU**, must be ended by a corresponding command, e.g. **ENDSETUP** or **ENDMENU**. It is indicated below where necessary.

How glossary entries are arranged

Glossary entries on script commands consist of three parts:

- **The name of the command** on the left side of the page. Related commands and other commands explained in the same glossary entry are listed underneath.
- **The command syntax** appears to the right of the command name. The syntax indicates the other types of information that are needed for the command to operate properly, and how the parts should be arranged.
- **The command definition** is found below the command syntax. The definition explains exactly what the command does, and gives examples of its use.

Syntax abbreviations

The explanations of command syntax use a number of terms to denote different types of information:

- **<value>**: indicates a number.
- **<string>**: indicates a text string within quotation marks.
- **<string exp>**: indicates either text within quotation marks or a variable that contains text (e.g. `user$` if the information it represents is, say, "Adam").
- **<variable>**: indicates either a persistent or non-persistent variable, e.g. `mailpass$` or `modeminit`.
- **<filename>**: indicates a filename within quotation marks. It is usually better to include the full path in filenames, e.g. `"C:\Documents\Logs\Log1"` rather than `"Log1"`.
- **<handle>**: indicates a numeric label you have assigned to a file you are opening. You can open up to 9 files, using the handles 1-9.
- **<label>**: indicates a script label.
- **<exp>**: indicates an expression, e.g. a condition in an **IF...THEN** command such as `'day$="Tuesday"'`.



If the syntax explanation only contains the command, e.g. **DRAIN**, then the command does not require any additional information when used in a script.

- Where the command syntax contains square brackets, the elements within brackets are optional.

Glossary

& see **SEND**

**// (DOUBLE
FORWARD-SLASH)**

//

Precedes a comment you have included to explain what part of a script does. Everything from the **//** to the end of the line is ignored. You can also put a comment after another command, e.g.

```
SEND "Fred","xmodem" // sends password
```

(There is no need to insert a colon.)

ALERT

ALERT <string exp>[,<string exp>]

Prompts the user (indicated by the text string) and waits for acknowledgement before continuing. For example,

```
ALERT "No carrier - try again later."
```

You can **add** a second line to the alert dialog using an optional second string of text. For example,

```
ALERT "No carrier.", "Check modem is connected correctly and try again."
```

**AND
NOT
OR**

**AND
NOT
OR**

Use the **AND**, **NOT** and **OR** logical operators to link conditions in an expression. The following example only carries out the first **INFO** command if *both* conditions are met.

```
IF c>1 AND c<10 THEN
    INFO "Value is between 1 and 10"
ELSE
    INFO "Value is not between 1 and 10"
ENDIF
```

The following section of script defines **pass\$** and sends **pass\$** if *either* of the conditions are met.

```
IF usern$="Jerry" OR usern$="Sam" THEN
    pass$="disk14"
ELSE
    GOTO getpass
ENDIF
SEND pass$
```

You can use **NOT** to specify conditions that must *not* be met for a given action to be carried out.

```
IF username$="" AND NOT superuser$="Anton99" THEN
    GOTO getuserinfo
ELSEIF superuser$="Anton99" THEN
    GOTO superlog
ELSE
    GOTO getpass
ENDIF
```

The script jumps to **'getuserinfo'** only the variable **'username\$'** contains no information, and the variable **'superuser'** does *not* contain the information **'Anton99'**.

APPEND	see CAPTURE
AS	see OPEN
ASC	ASC (<string>) Returns the ASCII code of a character. If a whole string is entered between the brackets, ASC returns the code for the first character unless you specify a particular character using LEFT, MID or RIGHT. For example, <pre>INFO ASC ("H")</pre> returns the code 72.
BAUD	see SETUP
BEEP	BEEP <value>,<value> Activates the netpad's buzzer. The first value is the length of the note in 1/32s of a second, and the second indicates the pitch. For example, <pre>BEEP 16,300 ALERT "Connection dropped"</pre>
BREAK	BREAK Breaks out of a loop. The following example will only get as far as a=3. <pre>SET a=5 REPEAT SEND "<013>" a=a-1 if a=3 THEN BREAK UNTIL a=0</pre>
CALL	CALL <string expression> Stops running the current script and transfers control to a new one, without clearing any variables. The example starts 'script2.scr' running. <pre>CALL "C:\Documents\Scripts\script2.scr"</pre> When the called script ends, the Terminal emulation screen is shown. The program does not return to the calling script.
CAPTURE CAPTURE OFF APPEND DEBUG	CAPTURE <string>,[APPEND[,DEBUG]] Begins saving received characters to a specified file. Use CAPTURE OFF to stop the capture and save the file. For example, <pre>CAPTURE "C:\Documents\Logs\log15" ... CAPTURE OFF</pre> To add more captured information to a file that exists already, use the APPEND modifier. The example simply adds the new capture information onto the end of the old. <pre>CAPTURE "C:\Documents\Logs\log", APPEND</pre>

You may find **CAPTURE** useful when reading your email messages on a remote system. The following example sends "readmail" to the remote system - a command to 'read my messages' - and captures the resulting text. In this system, the remote machine signals the end of the mail by a return to the "ready:" prompt. If this happens within 5 minutes, the capture is terminated, if not, the script jumps to the 'fail5min' label.

```
CAPTURE "C:\Email\messages"  
SENDWAIT 600 "readmail" , "ready:" GOTO fail5min  
CAPTURE OFF
```



To store control characters in the captured file, e.g. <13> or <8>, use the **DEBUG** modifier. For example,

```
CAPTURE "C:\Scripting\log" , DEBUG
```

You can combine the two modifiers together by separating them with a comma. For example,

```
CAPTURE "C:\Captures\log2" , APPEND , DEBUG
```

CATCH

CATCH

**[INCLUDE/EXCLUDE],<string>,<variable>,<timeout>,<maxlength>
GOTO <label>**

Stores incoming information to the specified variable. For example,

```
CATCH INCLUDE, "OK" , R$ , 60 , 20 GOTO anerror
```

This captures incoming data to the variable R\$. The capturing will stop after a time of <timeout> (in this case 60 half-seconds), when the string "OK" is received, or when twenty characters have been received. The script jumps to the label anerror if the given string is not received before either the timeout value or the maximum number of characters is reached. Use **INCLUDE** if you want <string> to be included as part of the variable, or **EXCLUDE** if you do not.

CHAIN

CHAIN <string exp>

Starts a new script, clearing all variables first. For example,

```
CHAIN "C:\Documents\Scripts\script3.scr"
```

starts 'script3.scr' running, whilst clearing all the variables set up in any previous scripts.

CLOSE

see **OPEN**

CLS

CLS

Clears the terminal screen.

COLLATE

COLLATE (<string>)

Collates a string.

CONNECT	CONNECT <value> GOTO <label> Waits for the current handshaking method to allow transmission. The period to wait is specified in half seconds. The following example waits 10 seconds for the connection to be established. If it happens within this time, the script jumps to the next command ("send"), otherwise it moves to the 'again' label. <pre>CONNECT 20 GOTO again SEND "Hello"</pre>
CONTINUE	CONTINUE Makes the script jump to the start of REPEAT...UNTIL or WHILE...DO...ENDWHILE loop.
COPY	COPY <filename>,<filename> Copies a file, from the location specified in the first string expression to the location specified in the second. The following example creates an identical copy of the file 'Logfile3' in the specified directory on the D: drive. <pre>COPY "C:\Logs\Logfile3", "D:\Documents\Backups"</pre>
DATA	see SETUP
DEBUG	see CAPTURE
DELETE	DELETE <filename> Deletes a file. For example, <pre>DELETE "C:\Logs\Logfile3"</pre>
DIR	DIR (<string exp>) Returns a directory listing of files. To list all the files in a directory, use the following sequence of commands: <pre>X\$=DIR("C:\Documents\scripts*.*) while X\$<>" " do info X\$ X\$=dir ENDWHILE</pre> You can list specific files, say script files, by limiting the string expression. The example only lists files with the filename extension '.scr'. <pre>DIR("C:\Documents*.scr")</pre>
Do	see WHILE
DOWNLOAD	DOWNLOAD <string exp>,<string exp> Prepares the netpad to receive a file from the remote machine, using the protocol indicated by the second string expression. For example, <pre>DOWNLOAD "D:\Documents\letter1", "XModem"</pre>

The protocol is indicated by one of the following labels:

ASCII
XModem
YModem (batch)

Both XModem and ASCII protocols require you to specify a filename in the first string expression, as above. When receiving files using the Ymodem (batch) protocol, the filename is sent by the other machine, therefore a “blank text” string is entered. For example,

DOWNLOAD “”, “Ymodem (batch)”

DRAIN

DRAIN

Empties the “receive buffer” (the part of the memory where received characters are stored before being displayed). Any received characters that have yet to be processed will be discarded.

ELSE

see **IF**

ELSEIF

see **IF**

ENDMENU

see **MENU**

ENDQUERY

see **QUERY**

ENDSETUP

see **SETUP**

ENDWAIT

see **WAIT**

ENDWHILE

see **WHILE**

EOF

see **OPEN**

EXISTS

Returns a value of 1 or zero, depending on whether or not a file exists. If the file exists, the script carries out the commands specified by an if command. For example,

```
if exists "C:\Files\Log"=1 then
    goto gotlog
endif
goto nolog
```

checks to see if there is a file called ‘Log’ in the ‘Files’ folder. If there is (i.e. if exists returns a value of 1), the script jumps to the ‘gotlog’ label. If there is no such file (and exists returns 0), the script jumps to the ‘nolog’ label.

exists can also be used to assign a value to a variable. For example,

```
send exists "C:\Files\Log"
```

will send 0 or 1 depending on whether or not the file exists, and

```
gotlogfile$=exists "C:\Files\Log"
```

sets the value of the string according to the value returned by exists.

EXIT

EXIT

Stops running the script and returns to the Terminal emulation screen.
For example,

endsession:

```
INFO "End of session. Now hanging up."  
HANGUP  
EXIT
```

FAIL

see **SETUP**

FOLD

FOLD <string>

"Folds" a string (converts it to upper case, and removes any accents).
For example,

```
SEND FOLD Username$
```

FOR

see **OPEN**

FORGET

FORGET

Clears all variables.

GOTO

GOTO <label>

Jumps to the command following a specified label. For example,

```
GOTO prompt:  
...  
prompt:  
    WAIT 60  
        "Hello" GOTO gothello  
    ...  
ENDWAIT
```

The **GOTO** command makes the script jump straight to the **WAIT...ENDWAIT** command. The specified label can be anywhere in the script.

HANDSHAKE

see **SETUP**

HANGUP

HANGUP

Drops the DTR line for three seconds. If you are currently using a telephone line connection, your modem will hang up and drop the DCD line to the netpad. This command has the same effect as [Hangup](#) on the [Transfer](#) menu.

IF

ELSE

ELSEIF

THEN

ENDIF

IF <exp> THEN <command> [[ELSE] <command> [ELSEIF <exp> THEN <command>]] ENDIF

Allows you to specify a choice of instructions according to the conditions. **IF** requires both a **THEN** command, and an **ENDIF** command. For example,

```
IF Username$="" THEN  
    GOTO getuserinfo  
ELSEIF Username$="Bob" THEN  
    GOTO boblogon
```

```
ELSE GOTO getpass
ENDIF
```

checks to see if there is any information stored in the "Username" name. If there is no such information, the script jumps to the 'getuserinfo' label. If the name contains the information 'Bob', the script jumps to the 'boblogon' label. If the name contains something other than 'Bob', it jumps to the 'getpass' label.

You can link more than one condition to an IF command by using the OR and AND logical operators. For example,

```
IF Username$="" OR Password$="" THEN
    GOTO getuserinfo
```

jumps to the 'getuserinfo' label if *either* of the names has no information associated with it. On the other hand,

```
IF Username$="" AND Address$="" THEN
    GOTO getuserinfo
```

jumps to the 'getuserinfo' label only if *both* of the names have no associated information.

INFO

INFO <string exp>

Displays a message in the terminal screen, e.g. about the action currently being performed. For example,

```
INFO "Initialising modem and dialling out..."
```

You can use INFO messages to keep you informed about the progress of your connection.

INPUT

see OPEN

LEFT RIGHT

LEFT (<string exp>,<value>) RIGHT (<string exp>,<value>)

Extracts a specified number of characters from the furthest left or right part of string. For example,

```
SEND LEFT (logname$,3)
```

will send "sec" if the information stored as logname\$ is "secondlog". Similarly,

```
SEND RIGHT (logname$,3)
```

would send "log".

LEN

LEN <variable>

Returns the length of a string. For example,

```
namelen=LEN (name$)
```

will assign a value of 5 to 'namelen' if the string "Anton" is stored as name\$.

LOADNAMES SAVENAMES

LOADNAMES <string exp>

Loads a given names file, so that you can use its information in the current script. For example,

```
LOADNAMES "C:\Documents\Comms\BBSnames"
```

SAVENAMES saves all the current names information in a specified file. For example,

SAVENAMES "C:\Documents\Names\Newnames"

LOWER

see **UPPER**

MENU

ENDMENU

MENU <string exp>
 <string exp> **GOTO** <label>
 <string exp> **GOTO** <label>
 ...
ENDMENU

Offers a menu of options to the user. Each option jumps to a particular label when it is chosen. For example,

```
MENU "Choose action"  
    "Connect to server" GOTO logon  
    "Upload file" GOTO sendfile  
    "Download file" GOTO downl  
    "Exit" EXIT  
ENDMENU
```

creates a menu with the heading "Choose action" from which the user can select one of four actions (e.g. "Upload file" etc). The script then carries out the command next to the menu item, e.g. jumping to sendfile for the Upload file menu item. Menu items can only followed by **GOTO** or **EXIT** commands. **MENU** requires a matching **ENDMENU** command.

MID

MID (<string exp>,<value>[,<value>])

Extracts part of a string or variable, beginning a specified number of characters (indicated by the first value) from the left. For example,

```
logfile=$"C:\files\log1"  
INFO "..." & MID (logfile$,9)
```

will display "...log1" in the terminal screen.

The optional, second value allows you to specify how many letters are to be extracted. For example,

```
SEND MID ("password15",4,4)
```

will just send the characters 'word'.

MOVE

MOVE <filename>,<filename>

Moves a file from the location specified in the first string expression to the location specified in the second. For example,

```
MOVE "C:\Logs\Capture","C:\Email"
```

NOT

see **AND**

ON ERROR GOTO

ON ERROR OFF

ON ERROR GOTO <label>

Makes the script jump to the specified label if an error occurs. The line

```
ON ERROR OFF
```

returns Comms to normal error handling.

OPEN

INPUT

OUTPUT

OPEN <filename> FOR [INPUT/OUTPUT/APPEND] AS <handle>
 READ <length>,<variable>,<handle>
 WRITE <handle>,<string>

AS
FOR
READ
WRITE
EOF
CLOSE

EOF <handle>
CLOSE <handle>

Opens files and enables reading from, and writing to, them. The "handle" part of the command should be a number corresponding to one of the following:

To read a file,

Its really meant for text files...

When you open a file, you assign it a 'handle' that is then used to refer to that file. You can open up to 9 files at once, using handles from 1 to 9.

For example, to open a file called 'source' and copy it 16 bytes at a time to a file called 'dest' you could run the following (from memory):

```
OPEN "C:\Source" FOR INPUT AS 1
OPEN "C:\Dest" FOR OUTPUT AS 2

WHILE NOT EOF(1) DO
  READ 1,A$,16 REM Reads 16 bytes from file (or less if less remain)
  WRITE 2,A$ REM Writes contents of A$ to file 2
ENDHILE

CLOSE 1
CLOSE 2
```

OR

see **AND**

OUTPUT

see **OPEN**

QUERY
ENDQUERY
QUERYOK

QUERY <string>
 <string>,<variable>
 <string>,<variable>
ENDQUERY

Prompts the user for information that is then stored as a name. For example,

```
QUERY "Enter details"
  "Username:" , myname$
  "City" , city$
  "Password" , pass$
ENDQUERY
```

creates a dialog where the user can enter information such as "Username" etc. This information is then stored with the name that follows, e.g. "myname". Up to six items can be prompted for in a **QUERY** dialog. The string that follows **QUERY** is the title of the dialog. **QUERY** requires a matching **ENDQUERY** command.

QUERYOK allows the script to act differently depending on whether the **QUERY** dialog was exited by pressing **OK**, or by pressing Esc. For example,

```
QUERY "Enter details"
  "Username:" , myname$
  "City" , city$
  "Password" , pass$
ENDQUERY
IF QUERYOK THEN
  GOTO logon
```

```

ELSE
    GOTO cancelled
ENDIF

```

jumps to the 'logon' label if **OK** is pressed, but the 'cancelled' label if the query dialog is exited using Esc.

PARITY see **SETUP**

PORT see **SETUP**

READ see **OPEN**

RENAME **RENAME <filename>,<filename>**

Renames the file indicated in the first string expression to the name specified in the second. For example,

```

RENAME "C:\Documents\Names","Names2"

```

REPEAT
UNTIL
 see also **WHILE**

REPEAT
 <expression>
UNTIL <expression>

Repeats a command until a certain condition is met. For example,

```

SET a=5
REPEAT
    SEND "<013>"
    a=a-1
UNTIL a=0

```

Sends carriage returns and deducts 1 from the value of **a** until the value of **a** reaches 0.



The important difference between **REPEAT...UNTIL** loops and **WHILE...DO...ENDWHILE** loops is that **REPEAT...UNTIL** always executes the specified action at least once, and ceases to do so when a condition is met. By contrast, **WHILE...DO...ENDWHILE** checks for a condition *before* ever carrying out an action, so that if the condition is not met, the action will not be executed at all.

RESET **RESET**

Returns all communications settings back to their standard values. You may want to use this before a **SETUP** command, to be sure of the complete configuration. For example,

```

RESET
SETUP
    Baud=57600
    parity=Even
ENDSETUP

```

RIGHT see **LEFT**

SAVENAMES see **LOADNAMES**

SEND &

SEND <string exp> [;] [& <string exp>]

Sends data to a remote machine, usually for controlling a modem or for conveying information to a remote system. The example, sends a modem initialisation string.

```
SEND "atz"
```

You can add the character code for the **Enter** key to the send command by putting a semi-colon after the string, e.g.

```
SEND "mypassword";
```

To include control characters, such as carriage returns or line feeds, in a send string, use angled brackets. For example,

```
SEND "mail<13><10>"
```

sends a carriage return (control character number 13) and then a line feed (control character number 10) after 'mail'. You can send more than one string at a time by using the & operator. For example,

```
SEND "john" & "password29"
```

SEENDBREAK

SEENDBREAK

Sends a "break" (i.e. transmits nothing for a short space of time) to the remote machine. The effect of this depends on the type of remote machine and the software it uses.

SENDWAIT

SENDWAIT <value> <string exp>, <string exp> GOTO <label>

Combines a **SEND** command with a **WAIT** command. For example,

```
SENDWAIT 60 "Anton" , "Password:" GOTO nopass  
GOTO pass
```

does exactly the same as:

```
SEND "Anton"  
WAIT 60  
"Password:" GOTO pass  
GOTO nopass  
...
```

The netpad sends "Anton", then waits for 30 seconds (60 half-seconds) for the remote machine to send "Password:". If "Password:" is received, the script jumps to the 'pass' label. If no "Password:" is received, the script moves on to the next command ('GOTO nopass'). The script only follows the **GOTO** command after "Password:" if "Password:" *isn't* received. **SENDWAIT** requires a matching **ENDWAIT** command.

SET

SET <variable>=<value>

Set assigns a variable or name a given value. For example,

```
SET me$="Albert"  
SET c1=5
```

Assigns the information "Albert" to the name 'me'. It also gives the variable **c1** a value of 5. You can then decrement this value to control the number of times an action is performed. For example,

```
setcount:  
SET c1=5  
sendcr:  
SEND "<13>"  
c1=c1-1
```

```

IF c1=0 THEN
    GOTO next
ELSE
    GOTO sender
ENDIF
next:
...
```

first gives 'c1' value of 5. The script then sends a carriage return character ("**<13>**") and deducts 1 from the value of **c1**. If **c1** still hasn't reached 0, it repeats this step. After it has sent 5 carriage returns, it will move on to the 'next:' label.

SETUP
ENDSETUP
BAUD
PORT
DATA
FAIL
HANDSHAKE
PARITY
STOP
TIMEOUT

SETUP
 BAUD [rate]
 PORT [COMM::0][IRCOMM::0]
 DATA [5][6][7][8]
 FAIL [DSR][DCD][PARITY]
 HANDSHAKE [DCD][RTSCTS][DSR][XONXOFF][NONE]
 PARITY [NONE][ODD][EVEN]
 STOP [1][2]
 TIMEOUT [timeout]
ENDSETUP

Sets any or all of the communications parameters. All of the above parameters can be set, to any of the options within square brackets. You can separate each of the parameters either with a colon, or by putting them on a new line. For example,

```

SETUP
BAUD=19200 : STOP 1
DATA=8
PORT="COMM::0"
ENDSETUP
```



For both **HANDSHAKE** and **FAIL**, you can set more than one of the choices by separating them with a comma. For example,

```

HANDSHAKE=XONXOFF,DCD
FAIL=DCD,DSR
```

Parameters you do not specifically set with **SETUP** remain unchanged. **SETUP** requires a matching **ENDSETUP** command.

STATUS

STATUS <string>

Displays an information message in the status bar at the bottom of the Terminal emulation screen. Use status messages to keep informed about the progress and state of your connection, e.g.

```
STATUS "Connected to BBS."
```

STOP

see **SETUP**

THEN

see **IF**

TIMEOUT

see **SETUP**

TRUE

TRUE

FALSE

FALSE

Returns 1 if a specified condition is true, and 0 if it is false. For example,

```
IF user$="bob" THEN
    f$=TRUE
ELSE
    f$=FALSE
ENDIF
SEND f$
```

sends 1 if the information stored as `user$` is "bob", and 0 if it is not.

UNTIL

see **REPEAT**

UPPER

see *also*:

LEFT
MID
RIGHT

UPPER <string exp>

Converts text to upper case. For example,

```
SEND UPPER username$
```

will send "CLIVE" even if the information stored as "`username$`" is "Clive". Similarly,

```
INFO LOWER prompt$
```

will send display the information in lower case.

You can combine **UPPER** with other text control commands such as **MID**, **LEFT**, **RIGHT** etc. For example,

```
STATUS UPPER LEFT (L$,1) & LOWER (MID(L$,1))
```

will display `L$` with a capital letter at the start and the rest in lower case.

VERSION

VERSION

Returns the version number of Comms.

WAIT

ENDWAIT

WAIT <value>

```
<string> GOTO <label>
<string> GOTO <label>
```

...

ENDWAIT

Pauses the script for a given number of half-seconds, and waits for one of several strings. For example,

```
WAIT 10
    "No carrier" GOTO tryagain
    "Hangup" GOTO closeconn
ENDWAIT
SEND "ATDT91011992220"
```

pauses for 5 seconds before proceeding with the next command in the script ('**SEND**'). If, during that time, the netpad receives "No carrier" from the remote machine, it will carry out the '**GOTO tryagain**' command. Similarly, if the netpad receives "Hangup" during the 5 seconds, the script will jump to the '**closeconn**' label. **WAIT** requires a matching **ENDWAIT** command.

WHILE

ENDWHILE
DO

WHILE <exp> DO

...

ENDWHILE

Carries out a given command while a certain condition is met. For

example,

```
WHILE R$<>"OK" DO  
  SEND "AT"  
ENDWHILE
```

sends the "AT" string as long as R\$ is not equal to "OK".



The important difference between **REPEAT...UNTIL** loops and **WHILE...DO...ENDWHILE** loops is that **REPEAT...UNTIL** always executes the specified action at least once, and ceases to do so when a condition is met. By contrast, **WHILE...DO...ENDWHILE** checks for a condition *before* ever carrying out an action, so if the condition is not met, the action will not be executed at all.

WRITE

see **OPEN**

Index

- // (double forward slash)
 - in scripts, 32
- adding a line feed to carriage returns
 - incoming, 4, 14
 - outgoing, 4, 14
- appending information to a capture file, 9
- Baud rate, 3
- CALL command, 32
- CAPTURE command
 - with APPEND, 20
 - with DEBUG, 20
 - with OFF, 20
- capturing information, 9, 10
- carriage return codes, 4
- clearing the display, 4
- communication settings, 3
- CONNECT message, 6
- connecting to another machine, 5
 - directly, 5
 - to a PC, 5
 - via a modem link, 6
- connecting using Windows 95, 5
- connecting with another machine
 - via infrared, 7
- copying and pasting, 5
- data bits, 3
- DCD handshaking, 3
- Debug option for capturing information, 9
- dialling, 6
- Docking cable, 1
- DOWNLOAD command, 22
- DRAIN command, 22
- DSR/DTR handshaking, 3
- ELSE command, 32
- ELSEIF command, 32
- ending a connection, 7
- EXISTS command, 32
- EXIT command, 23, 33
- file conversion. *See* PsiWin 2
- file extensions, 9
- file transfer protocols
 - Ascii, 8
 - XModem, 8
 - YModem (batch), 8
- flow control. *See* handshaking
- , 9
- , 9
- GOTO command, 33
- handshaking, 3
- hangup. *See* ending a connection
- HANGUP command, 23, 33
- hardware handshaking. *See* RTS/CTS handshaking
- HyperTerminal, 5
- IF command, 33
- infrared comms port, 7
- line feed codes, 4
- loading communication settings, 6
- local echo, 2, 14
- MENU command, 33
- modem initialisation, 6
- modem speed, 3, 6
- names, 13
 - in scripts, 13
 - loading, 13
 - password protecting, 14
 - reserved, 14
 - saving, 13
 - setting up, 13
- Offline message, 2
- ON command, 34
- parity, 3
- paste to host, 5
- pausing the display, 6
- plain text files, 9
- 'Port active' command, 2
- PsiWin 2, 1
- QUERY command, 34
- receiving
 - a single file, 9
 - multiple files, 9
- Remote link, 2
- RESET command, 34
- RTC/CTS handshaking, 3
- SAVENAMES command, 34
- saving communication settings, 5
- scripting language
 - commands, 15
 - alphabetical list, 17
 - QUERY, 13
 - summary, 15
- scripts, 10
 - automating connections, 10
 - calling other scripts, 11
 - checking syntax, 10
 - creating, 10
 - editing, 10
 - end-commands, 12
 - ending with carriage return, 11
 - labels, 11
 - names. *See* names
 - prompting for information, 13
 - running, 11
 - saving, 10
 - script commands, 11
 - sending text strings, 12
 - stopping, 11
 - using filenames and folders, 12
- scroll bars
 - hiding, 4
 - showing, 4
- SEND command, 28, 34
- sending
 - a single file, 8
 - multiple files, 8
- SENDWAIT command, 35
- SET command, 29
- SETUP command, 35
- 'Start script' command. *See* scripts:running
- starting Comms, 2
- , 2
- STATUS command, 36

stop bits, 3
'Stop script' option. See
 scripts:stopping
stopping a file transfer, 9
teletype emulation, 4
terminal emulation, 4
Terminal emulation
 screen, 2

THEN command, 36
transferring Ascii
 information, 9
transferring files between
 machines, 8
translate codes, 14
Translate codes, 2

TTY. See teletype
 emulation
UPLOAD command, 36
Use comms port, 3
VT100 emulation, 4
WAIT command, 36
Xon/Xoff handshaking, 3